基础实验：实验报告

姓名：程宣赫　　学号：2023202250

2024 年 10 月 3 日

1　问题分析

在这个民航订票的编程实验中，整个问题可以被抽象成链表或顺序表的构建和问题中一些基本操作，如初始化、排序、查询及插入和删除的代码抽象。

给定的航班数据可以用顺序表（结构体数组）和链表两种不同的存储方式管理。顺序表比较适合频繁进行数据访问而较少插入、删除操作的场景，例如实现栈、队列等；而链表比较适合频繁进行插入、删除操作而较少访问特定位置元素的场景，例如实现动态数据集合。因此，综合实验后续的问题看来，我在对航班数据的抽象上使用了链表进行储存。

而基本操作的代码抽象总结分类下来则有以下几点：（1）数据初始化：从 CSV 文件中读取航班数据，并将其存储在顺序表中。为数据提供良好的存储结构，以便后续操作；（2）数据排序：通过启航机场 ID、抵达机场 ID 和航程时长对航班进行排序。这涉及使用高效的排序算法来处理链表或数组；（3）数据查询：包括对基础费用为素数的航班的查找、鞍点的查找以及按飞机 ID 构建链表的功能；（4）插入与删除：根据用户输入在原链表中删除某种飞机机型，并将删除后的航班数据存储在新的链表中，再对新链表进行操作。

2　数据结构设计与实现

2.1　ADT 设计

- 数据对象：航班（Flight）结构体，即链表中数据域储存的数据，这部分数据包含以下字段：

1 Flight ID（航班 ID）：唯一标识航班

2 Departure Date（启航日期）

3 Intl/Dome（航班类型，国际或国内）

4 Departure/Arrival Airport（启航/抵达机场 ID）

5 Airplane ID（飞机 ID）

6 Airfares/Peakrates/Offrates（基础/高峰/低谷费用）

- 数据关系：线性关系（链表）
- 基本操作：初始化、排序、查询、插入、删除

2.2 ADT 的物理实现

- C++ 语言代码展示举例

```
#include <iostream>
#include <string>
#include <fstream>
#include <limits>
#include <sstream>
#include <iomanip>
#include <chrono>

typedef struct
{
    int Flight_id;//航班ID
    int Departure_date;//启航日期
    bool Flight_type;//航班类型
    string Flight_no;//航班编号
    int Departure_airport;//启航机场ID
    int Arrival_airport;//抵达机场ID
    float Departure_time;//启航时间
    float Arrival_time;//抵达时间
    int Airplane_id;//飞机ID
    int Airplane_model;//飞机机型
    int Air_fares;//基础费用
    int Peak_rates;//旺季价格
    int Off_rates;//淡季价格
}Flight;

class Flight_node
{
    public:
        Flight flight_node_data;
        Flight_node* next;
        Flight_node* prev;

        Flight_node(): flight_node_data(), next(nullptr), prev(
           nullptr)
        {}
        Flight_node(Flight x): flight_node_data(x), next(nullptr),
           prev(nullptr)
        {}
};
```

```
class Data_List
{
    private:
        Flight_node* head;//头指针
        Flight_node* tail;//尾指针
        int size;//链表长度

    public:
        //初始化链表
        Data_List() : head(nullptr), tail(nullptr), size(0);
        //销毁链表释放内存
        ~Data_List() ;
        //清空链表
        void clear_list();
        //检查链表是否为空
        bool List_Empty();
        //添加节点
        void add(Flight x);
        //得到头结点
        Flight_node* get_head();
        //数据初始化，操作1
        void Initialize_Data_List();
        //数据排序，操作2
        void Sort_Data_List();
        //数据查询3.1,输出所有基础费用为素数的航班ID
        void Search_Date_List1();
        //数据查询3.2
        void Search_Date_List2();
        //数据查询3.3
        void Search_Date_List3(Data_List& list2);
        //新链表制造4.1
        void Delete_Airplane_model(Data_List& data_list3);
```

3 算法设计与实现

3.1 算法设计

该实验除去读入文件初始化链表外需要核心考虑的重点算法有以下三个：多重排序（冒泡排序）、鞍点查找、构建链表

实验问题中的操作 2 与 3.3 实际上都是一个排序问题，区别只是在于 2 是单纯的对已有链表进行排序，而 3.3 是新构建一个链表再对新链表进行排序，但其本质还是一样的。于是我选择

了多重多条件嵌套排序，且使用冒泡排序的方法进行排序，但显而易见的问题是，该方法较为复杂，且冒泡排序的时间复杂度较高，更优解其实可以使用快排对链表进行排序，但我对链表的快排实现不是很清楚，因此最终还是选择了冒泡排序。

在题目中定义的鞍点是一个行中的最大值，列中最小值的元素，因此我进行了两次对链表的遍历，第一次确定三个价格列中的最低值，随后再对链表进行一次遍历，找到满足鞍点要求的元素，并返回它数据域中的航班 ID。由于该算法核心知识对链表进行了两次遍历，所以时间复杂度较简单，暂时也想不到什么其他优化的空间

操作 4 分支下的 4.2、4.3、4.4 其实只是重复前面的操作，所以可以重复使用前面的代码。而 3.3 和 4.1 主要包含的是新链表的构造，我所想的是先遍历整个链表，判断每个节点的飞机模型是否与给定的一致，一致的就通过链表内的加入节点操作加入到新链表中，4.1 对于原链表的删除操作则直接在 4.1 操作中实现，而不是使用链表内部的 delete 操作，避免时间和空间的无意义消耗。

由于在数据结构的选择上，我选择了链表，所以对算法的空间复杂度较为友好。

- 时间复杂度分析：

1 数据初始化：$O(n)$

2 排序：$O(n^2)$

3 鞍点查找：$O(n)$

4 删除和插入：$O(n)$

- 空间复杂度分析：

1 数据初始化：$O(n)$

2 排序：$O(1)$

3 鞍点查找：$O(1)$

4 删除和插入：$O(n)$

3.2 算法实现

- 链表的冒泡排序

```
bool swapped = true;
Flight_node* current = head;
Flight_node* last_node = nullptr;

while(swapped)
{
    swapped = false;
    current = head;
```

```
while(current -> next != last_node)
{
    if(current -> flight_node_data.Departure_airport
       < current -> next -> flight_node_data.
       Departure_airport)
    {
        swap(current -> flight_node_data , current ->
            next -> flight_node_data);
        swapped = true;
    }

    else if(current -> flight_node_data.
       Departure_airport == current -> next ->
       flight_node_data.Departure_airport
            && current -> flight_node_data.
               Arrival_airport < current -> next ->
               flight_node_data.Arrival_airport)
    {
        swap(current -> flight_node_data , current ->
            next -> flight_node_data);
        swapped = true;
    }

    else if(current -> flight_node_data.
       Departure_airport == current -> next ->
       flight_node_data.Departure_airport
            && current -> flight_node_data.
               Arrival_airport == current -> next ->
               flight_node_data.Arrival_airport)
    {
        float current_duration = current ->
           flight_node_data.Arrival_time - current ->
            flight_node_data.Departure_time;
        float next_duration = current -> next ->
           flight_node_data.Arrival_time - current ->
            next -> flight_node_data.Departure_time;
        if(current_duration > next_duration)
        {
            swap(current -> flight_node_data ,
               current -> next -> flight_node_data);
            swapped = true;
        }
```

```
            }
            current = current -> next;
        }
        last_node = current;
    }
```

- 鞍点查找

```
    Flight_node* current = head;
    int min_air_fares = 114514;
    int min_peak_rates = 114514;
    int min_off_rates = 114514;

    //遍历链表分别求三列的最小值
    while(current != nullptr)
    {
        if(current -> flight_node_data.Departure_airport ==
           departure_airport && current -> flight_node_data.
           Arrival_airport == arrival_airport)
        {
            if(current -> flight_node_data.Air_fares <
               min_air_fares)
                min_air_fares = current -> flight_node_data.
                   Air_fares;

            if(current -> flight_node_data.Peak_rates <
               min_peak_rates)
                min_peak_rates = current -> flight_node_data.
                   Peak_rates;

            if(current -> flight_node_data.Off_rates <
               min_off_rates)
                min_off_rates = current -> flight_node_data.
                   Off_rates;
        }
        current = current -> next;
    }

    //再次遍历链表，找到满足鞍点的航班ID
    current = head;
    while(current != nullptr)
    {
```

```
if(current -> flight_node_data.Departure_airport ==
   departure_airport && current -> flight_node_data.
   Arrival_airport == arrival_airport)
{
    if(current -> flight_node_data.Air_fares ==
       min_air_fares
    && current -> flight_node_data.Air_fares >
       current -> flight_node_data.Peak_rates
    && current -> flight_node_data.Air_fares >
       current -> flight_node_data.Off_rates)
    {
        count++;
        cout << "The flight ID of saddle point " <<
           count << " based on Air_fares is: " <<
           current -> flight_node_data.Flight_id <<
           endl;
    }

    if(current -> flight_node_data.Peak_rates ==
       min_peak_rates
    && current -> flight_node_data.Peak_rates >
       current -> flight_node_data.Air_fares
    && current -> flight_node_data.Peak_rates >
       current -> flight_node_data.Off_rates
    )
    {
        count++;
        cout << "The flight ID of saddle point " <<
           count << " based on Peak_rates is: " <<
           current -> flight_node_data.Flight_id <<
           endl;
    }

    if(current -> flight_node_data.Off_rates ==
       min_off_rates
    && current -> flight_node_data.Off_rates >
       current -> flight_node_data.Air_fares
    && current -> flight_node_data.Off_rates >
       current -> flight_node_data.Peak_rates
    )
    {
        count++;
        cout << "The flight ID of saddle point " <<
```

```
                    count << " based on Off_rates is: " <<
                    current -> flight_node_data.Flight_id <<
                    endl;
            }
        }
        current = current -> next;
    }

    //如果没找到鞍点，输出提示信息
    if(count == 0)
        cout << "No saddle point found." << endl;
```

- 数据的插入与删除

```
    Flight_node* current = head;
    while(current != nullptr)
    {
        if(current -> flight_node_data.Airplane_model ==
           airplane_model)
        {
            //加入新链表
            data_list3.add(current -> flight_node_data);
            data_list3.size++;
            cout << current -> flight_node_data.Flight_id <<
               " has been added to the new list(data_list3)."
                << endl;

            //分头结点，尾结点，中间节点分别考虑删除原链表中
               的节点
            if(current == head)//头
            {
                head = current -> next;
                if(head != nullptr)
                {
                    head -> prev = nullptr;
                }
                else
                {
                    tail = nullptr;
                }
            }
            else if(current == tail)//尾
```

```
                {
                    tail = current -> prev;
                    tail -> next = nullptr;
                }
                else //中间
                {
                    current -> prev -> next = current -> next;
                    current -> next -> prev = current -> prev;
                }
                delete current;
                this -> size--;
            }
            current = current -> next;
        }
```

4 实验环境

- 硬件环境:

1 CPU:12th Gen Intel(R) Core(TM) i7-12700H 2.70 GHz

2 内存: 16GB

- 软件环境:

1 操作系统: Microsoft Windows 11 家庭中文版

2 编程语言: C++

3 IDE 及版本: Visual Code Studio 1.93.1

5 实验结果与分析

某个操作后，展示并分析对应实验结果。
展示图片 Fig 1如下。

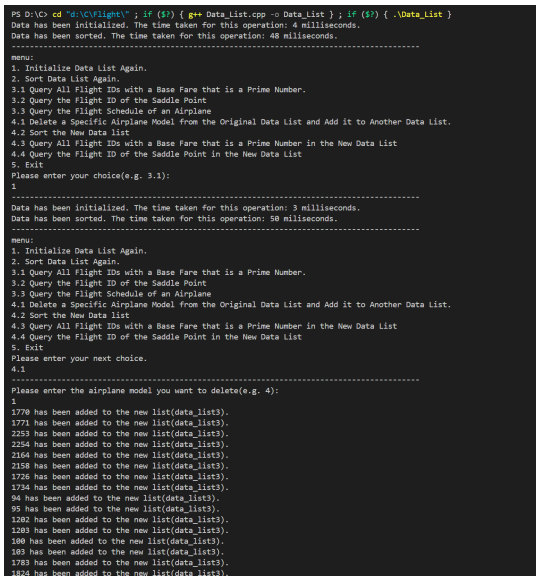
PS D:\C> cd "d:\C\flight\" ; if ($?) { g++ Data_List.cpp -o Data_List } ; if ($?) { .\Data_List }
Data has been initialized. The time taken for this operation: 4 milliseconds.
Data has been sorted. The time taken for this operation: 48 milliseconds.
menu:
1. Initialize Data List Again.
2. Sort Data List Again.
3.1 Query All Flight IDs with a Base Fare that is a Prime Number.
3.2 Query the Flight ID of the Saddle Point
3.3 Query the Flight Schedule of an Airplane
4.1 Delete a Specific Airplane Model from the Original Data List and Add it to Another Data List.
4.2 Sort the New Data list
4.3 Query All Flight IDs with a Base Fare that is a Prime Number in the New Data List
4.4 Query the Flight ID of the Saddle Point in the New Data List
5. Exit
Please enter your choice(e.g. 3.1):
1
Data has been initialized. The time taken for this operation: 3 milliseconds.
Data has been sorted. The time taken for this operation: 50 milliseconds.
menu:
1. Initialize Data List Again.
2. Sort Data List Again.
3.1 Query All Flight IDs with a Base Fare that is a Prime Number.
3.2 Query the Flight ID of the Saddle Point
3.3 Query the Flight Schedule of an Airplane
4.1 Delete a Specific Airplane Model from the Original Data List and Add it to Another Data List.
4.2 Sort the New Data list
4.3 Query All Flight IDs with a Base Fare that is a Prime Number in the New Data List
4.4 Query the Flight ID of the Saddle Point in the New Data List
5. Exit
Please enter your next choice.
4.1
Please enter the airplane model you want to delete(e.g. 4):
1
1770 has been added to the new list(data_list3).
1771 has been added to the new list(data_list3).
2253 has been added to the new list(data_list3).
2254 has been added to the new list(data_list3).
2164 has been added to the new list(data_list3).
2158 has been added to the new list(data_list3).
1726 has been added to the new list(data_list3).
1734 has been added to the new list(data_list3).
94 has been added to the new list(data_list3).
95 has been added to the new list(data_list3).
1202 has been added to the new list(data_list3).
1203 has been added to the new list(data_list3).
100 has been added to the new list(data_list3).
103 has been added to the new list(data_list3).
1783 has been added to the new list(data_list3).

图 1: 图片展示举例